

LNbitsBox: Self-Hosted Lightning, Ready to Build On

Framing

- Audience: developers, hackers, tech enthusiasts working around Bitcoin, Lightning, Nostr, privacy, and decentralization tools.
- Tone: fast, practical, builder-friendly, and demo-led.
- Speaker identity: BlackCoffee.
- This replaces the previous talk content entirely. No “about me” or “about LNbits” section.
- Source context: Freedom Tech Summit describes its audience and subject matter around hands-on freedom tech, Bitcoin, Lightning, Nostr, privacy, and decentralization tools:
<https://freedomtechsummit.com/>

Slide 1 - What Is LNbitsBox?

- LNbitsBox is a self-hosted Lightning appliance.
- It is a Raspberry Pi image that boots into a working LNbits server.
- The critical parts are already wired together:
 - First-run setup wizard
 - LNbits with Phoenixd, Ark(ade) or Spark Lightning funding sources
 - Tor hidden service
 - Clearnet URL
 - Admin control panel
 - Wi-Fi config
 - OTA updates
 - Manual and rolling encrypted backup and recovery
 - Local HTTPS with installable CA
- Visual flow: Flash SD card -> Boot Pi -> Configure -> Run LNbits.

Speaker cue

“Think of it as LNbits you can flash, boot, configure, and hand to someone without asking them to become a Linux sysadmin first.”

Slide 2 - Why?

- Because sovereign self-hosting can be easy.
- LNbitsBox gives you a fully self-contained Lightning appliance that is easy to set up, use, and maintain.
- Use cases:
 - Run your own LNbits instance at home
 - Put it in a shop, classroom, hackerspace, or workshop
 - Build a merchant, Lightning, or project box without depending on a cloud account for the core experience
- Core value:
 - No cloud account required for the core experience
 - Tor support for private reachability
 - Admin panel for status, services, Wi-Fi, updates, and tunnels
 - Built for real-world demos: reset, reconfigure, update, recover
- Visual structure: three columns for Personal node, Workshop device, Merchant / project box.

Speaker cue

“The goal is not to hide the stack. The goal is to make the first successful boot boring enough that people can start building.”

Slide 3 - Why NixOS?

- Because the box should be reproducible, not artisanal.
- NixOS makes the appliance declarative:
 - Services
 - Ports
 - Files
 - Packages
 - Boot behavior
 - Update flow
- Why it matters:
 - Same config builds the same system image
 - Easier OTA updates via immutable system generations
 - Rollback-friendly when updates go wrong
 - Services are explicit: LNbits, Spark sidecar, Caddy, Tor, admin app
 - Great fit for appliances: define the state machine once, ship it as an image
- Visual flow: flake.nix -> SD image -> Raspberry Pi -> reproducible LNbitsBox.

Speaker cue

"With NixOS, the product is not just the app. The product is the whole machine state."

Demo - Go Straight To The Box

- Keep the transition short. The slides exist to establish what people are about to see.
- Demo sequence:
 1. Configure the box
 2. Show the control panel
 3. Open inbitsbox.com / project site for download and docs

Closing line

"That is the whole point: flash it, boot it, make it reachable, and start building."